

LoRa Documentation

Radiolib Reference for SX126x: https://jgromes.github.io/RadioLib/class_s_x126x.html

Radiolib Reference for SX1262: https://jgromes.github.io/RadioLib/class_s_x1262.html

Pico-LoRa-SX1262 module page: <https://www.waveshare.com/wiki/Pico-LoRa-SX1262-868M>

SX1262 Technical Documentation:

https://semtech.my.salesforce.com/sfc/p/#E0000000JelG/a/RQ000008nKCH/hp2iKwMDKWl34g1D3LBf_zC7TGBRlo2ff5LMnS8r19s

1. Hardware Pinout

Pinout

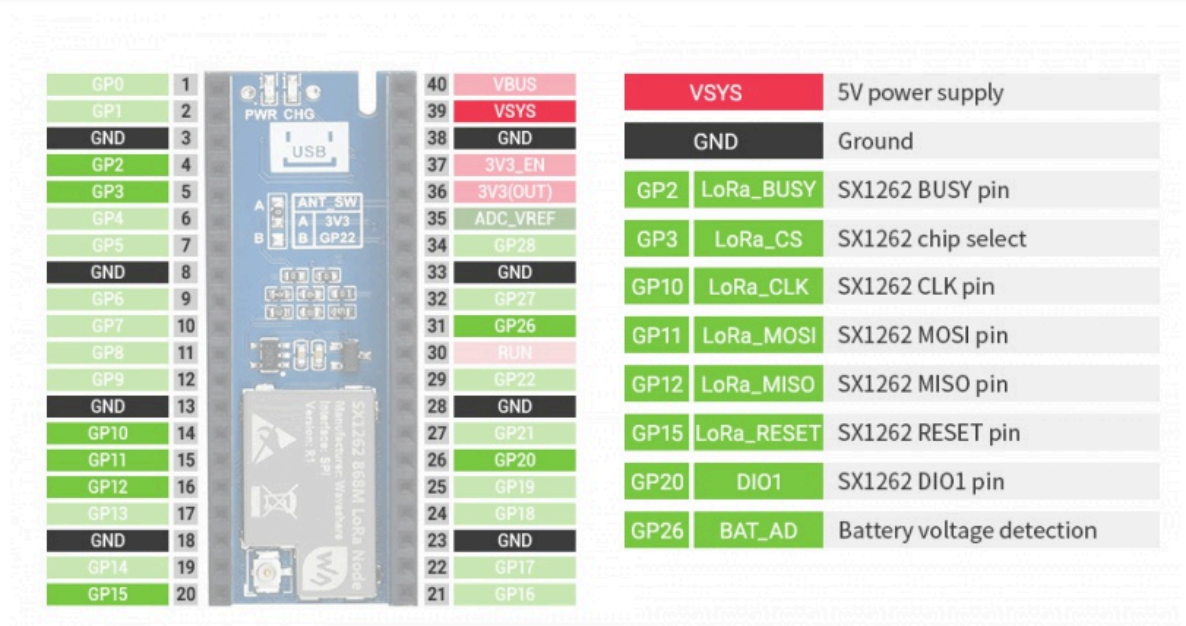


Figure.1 Pin layout of the LoRa module

The LoRa module talks to the pico using SPI. Notice that the SPI pins used belong to the *spi1* port on the pico, therefore *spi1* should be assigned and initialized as the SPI port.

GPIO 15 is the RESET for the LoRa module and should be pulled *HIGH* for the module to function.

GPIO 20 is the DIO1 pin used by the LoRa module to signal the pico module that a transmitting or receiving operation has been done. It allows interrupt based transmission or reception.

2. Initialization

Radiolib was used for the LoRa operation.

Class *PicoHal* was used to store SPI port and pins. It also has Arduino styled HAL functions as Radiolib was initially designed for Arduino. We successfully constructed the class using the following line:

```
PicoHal* hal = new PicoHal(SPI_PORT, SPI_MISO, SPI_MOSI, SPI_SCK);
```

Parameter	Value	Comment
SPI_PORT	spi1	Spi port used by pico
SPI_MISO	12	Master In Slave Out (RX)
SPI_MOSI	11	Master Out Slave In (TX)
SPI_SCK	10	Clock

The port and pins above are configured according to the hardware layout in the previous section.

Class SX1262 has functions to configure and command the SX1262 module. We constructed the class using the following line.

```
SX1262 radio = new Module(hal, RFM_NSS, RFM_DIO1, RFM_RST,  
RFM_BUSY);
```

Parameter	Value	Comment
hal	Pointer of constructed PicoHal	
RFM_NSS	3	SPI chip select
RFM_DIO1	20	Interrupt pin signaling operation done
RFM_RST	15	RESET pin
RFM_BUSY	2	

With RFM_RST pin pulled up and spi module initialized (*spiBegin()* from PicoHal used in the code), the begin() function of SX1262 class could be called to initialize LoRa communication.

```
int state = radio.begin(FREQUENCY,
                        BANDWIDTH,
                        SPREADING_FACTOR,
                        CODING_RATE,
                        SYNC_WORD,
                        OUTPUT_POWER,
                        LORA_PREAMBLE_LEN,
                        TCXO_VOLTAGE);
```

The parameters contain various transmission parameters. Different transmission configurations have different bit rate and sensitivity. The most important ones are bandwidth and spreading factor (Page 19 and 20 of SX1262 manual)

BR_L	Bit rate LoRa	Min. for SF12, BW_L = 7.8kHz Max. for SF5, BW_L = 500kHz	0.018	-	62.5 ⁶	kb/s
		BW_L = 10.4kHz, SF = 7	-	-134	-	dBm
		BW_L = 10.4kHz, SF = 12	-	-148	-	dBm
		BW_L = 125kHz, SF = 7	-	-124	-	dBm
		BW_L = 125kHz, SF = 12	-	-137	-	dBm
		BW_L = 250kHz, SF = 7	-	-121	-	dBm
		BW_L = 250kHz, SF = 12	-	-134	-	dBm
		BW_L = 500kHz, SF = 7	-	-117	-	dBm
		BW_L = 500kHz, SF = 12	-	-129	-	dBm
RXS_LB	Sensitivity LoRa, Rx Boosted gain, see Section 9.6 "Receive (RX) Mode" on page 62, split RF paths for Rx and Tx, RF switch insertion loss excluded					

Other parameters are further explained in page 38 of the SX1262 manual.

This link is an official calculator to try out the parameters
<https://www.semtech.com/design-support/lora-calculator>

It's also possible to use *beginFSK()* to communicate with FSK modulation instead. It has higher data rate but worse range and resistance to interference compared to LoRa.

```
radio.setDio1Action(setFlag);
```

This sets the interrupt handler. *setFlag* is the handler we declared in the beginning of the program with a volatile boolean variable *operationDone*. We use this variable to prevent us from reading or sending incomplete garbage, and to switch between TX and RX.

```
volatile bool operationDone = false;
void setFlag(void) {
    operationDone = true;
}
```

```
#if defined(INITIATING_NODE)
// send the first packet on this node
printf("[SX1262] Sending first packet ... ");
transmissionState = radio.startTransmit("Hello World!");
transmitFlag = true;
#else
// start listening for LoRa packets on this node
sleep_ms(2000);
printf("[SX1262] Starting to listen ... ");
state = radio.startReceive();
if (state == RADIOLIB_ERR_NONE) {
    printf("success!");
} else {
    printf("failed, code ");
    while (true) { sleep_ms(10); }
}
#endif
```

SX1262 is a transceiver, it cannot do TX and RX at the same time. To communicate in two directions, the program was adapted from the Ping Pong example code. The receiving module will start to transmit after it has received a packet, similarly, the transmitting module will start to listen after it has transmitted a module.

The block of code bounded by if-condition above is used to determine which node initiates the communication and send the first packet.

3. Operation

```
while(1){  
  
    if (operationDone){  
        operationDone = false;  
  
        if(transmitFlag) {  
            // the previous operation was transmission, listen for response  
            // print the result  
            if (transmissionState == RADIOLIB_ERR_NONE) {  
                // packet was successfully sent  
                printf("transmission finished!\n");  
            } else {  
                printf("failed, code ");  
                printf("%d", transmissionState);  
            }  
        }  
  
        // listen for response  
        radio.startReceive();  
        printf("Listening \n");  
        transmitFlag = false;  
    }  
}
```

This is the main loop of the ping pong program.

operationDone is a status variable that could be set by interrupt on DIO1. If it's true then the SX1262 module is done with the previous operation and is ready for the next one.

transmitFlag is a status variable. If it's true then the previous operation was transmission, if it's false then the module was listening instead.

To receive, call radio.startReceive(); Then in the next time when operationDone is true, radio.readData() can be called, the parameters are the pointer to an uint8_t array and the length of that array.

To transmit, call `radio.startTransmit()`, inside is the string you want to send.